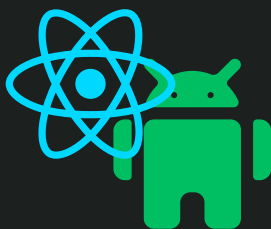


— por
Vinícius
Guedes



Bridge React Native

Conceitos de conexões nativas
entre **React Native e Android**



SOBRE O AUTOR

Sou o Vinícius Guedes especialista em React Native com mais de 8 anos de experiência no desenvolvimento de aplicativos móveis cross-platform e nativos. Ao longo de minha carreira, tenho atuado na criação e otimização de soluções móveis robustas, combinando conhecimento profundo em tecnologias modernas com uma abordagem prática e orientada para resultados. Minha expertise abrange desde a construção de interfaces de usuário sofisticadas até a integração de funcionalidades nativas complexas, que me renderam conhecimentos em palestras e desafios no campo do desenvolvimento móvel.

Este ebook é fruto de minha experiência e conhecimento na área, e reflete meu compromisso em capacitar desenvolvedores com técnicas e práticas avançadas para criar aplicações móveis de alta qualidade.

O QUE IREMOS APRENDER

Neste ebook, vamos explorar como estabelecer uma conexão eficaz entre o React Native e o código nativo em Android usando uma bridge. Iremos trabalhar com o Kotlin através de uma abordagem prática e detalhada, você irá aprender:

- **Chamar uma Função Simples:**
 - Objetivo: Aprender a expor e chamar funções básicas do código Kotlin a partir do React Native.
 - Detalhes: Vamos implementar uma função simples em Kotlin e chamá-la diretamente do JavaScript, demonstrando a integração básica entre os dois mundos.
- **Enviar Parâmetros para uma Função:**
 - Objetivo: Entender como passar parâmetros do React Native para o código Kotlin.
 - Detalhes: Iremos criar uma função para aceitar parâmetros e testar o envio de dados do React Native para o código nativo, evidenciando a comunicação bidirecional.
- **Retornar Valores de uma Função:**
 - Objetivo: Aprender a retornar valores do código nativo para o React Native.
 - Detalhes: Criaremos a lógica para que o código Kotlin retorne um valor para o JavaScript, utilizando Promises para lidar com a resposta assíncrona.
- **Criar um Listener de Eventos:**
 - Objetivo: Implementar e gerenciar eventos emitidos do código nativo para o React Native.
 - Detalhes: Iremos criar um listener no React Native para escutar eventos emitidos pelo código Kotlin e mostrar como reagir a esses eventos em tempo real.

SUMÁRIO

Introdução	5
Pré requisitos	6
Projeto	7
Módulo Nativo	10
Conexão React Native	17
Envio de Parâmetros	21
Retorno de Função	24
Event Listener	29
Resumo	34

INTRODUÇÃO

No universo do desenvolvimento de aplicativos móveis, o React Native se consolidou como uma das ferramentas mais poderosas para a criação de aplicativos cross-platform. Ele permite que os desenvolvedores escrevam um único código em JavaScript que roda tanto no Android quanto no iOS, otimizando o tempo de desenvolvimento e manutenção. No entanto, existem momentos em que você precisa acessar funcionalidades específicas de uma plataforma que não estão disponíveis diretamente no React Native. É aqui que o conceito de Bridge entra em ação.

A Bridge é uma ponte entre o código JavaScript e os módulos nativos, permitindo que você interaja diretamente com as APIs nativas do iOS ou Android. No caso do Android, ao construir uma bridge, você pode acessar recursos e funcionalidades que estão além do escopo do React Native, utilizando Java e/ou Kotlin. Isso oferece uma flexibilidade incomparável para criar soluções sob medida, como acesso a bibliotecas específicas do sistema, ou até mesmo otimizações de desempenho.

Compreender como construir uma bridge é uma habilidade essencial para qualquer desenvolvedor React Native que deseje se destacar. Embora o React Native ofereça soluções que cobrem a maior parte das necessidades de um projeto, saber integrar módulos nativos abre um mundo de possibilidades. Um desenvolvedor que domina a criação de bridges pode:

- Melhorar o desempenho de aplicativos que demandam processamento intensivo.
- Integrar funcionalidades que ainda não têm suporte oficial no React Native.
- Construir módulos reutilizáveis que podem ser aproveitados por outros desenvolvedores.

Este conhecimento também expande suas oportunidades de carreira, pois você será capaz de enfrentar desafios mais complexos e técnicos, tornando-se um desenvolvedor mais completo e preparado para projetos maiores e mais sofisticados.

Espero que você tenha um grande proveito deste eBook e que seja um diferencial para sua carreira. Este eBook vai guiá-lo passo a passo através desse processo, facilitando sua jornada para desbloquear todo o potencial do desenvolvimento nativo dentro do React Native. Vamos começar?

PRÉ REQUISITOS

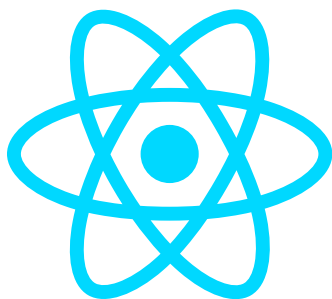
Antes de mergulharmos no processo de criação de bridges no iOS, é importante que você já tenha um entendimento sólido de alguns conceitos e ferramentas:

- **React Native:** Ambiente de desenvolvimento React Native configurado.
- **JavaScript:** Como os módulos nativos se comunicam via JavaScript, é essencial que você esteja confortável com essa linguagem.

O propósito deste curso é te fazer **entender os conceito de conexões bridge**. Para isso, escolhi usar o Expo Bare Workflow por se tratar da maneira "menos comum" de se criar um projeto de alta escala. Assim, quero te fazer entender que independe de React Native CLI ou Expo (desde que seja Bare Workflow) você consegue realizar conexões com módulos nativos.

Projeto React Native

CAPÍTULO 1



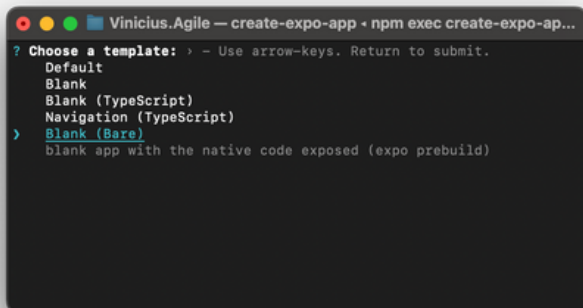
PROJETO REACT NATIVE

Na página anterior, explicamos o motivo de se usar o Expo, então vamos agora criar nosso projeto.

Em seu terminal navegue até a pasta em que deseja criar o seu app, e execute o comando abaixo para criarmos nosso app.

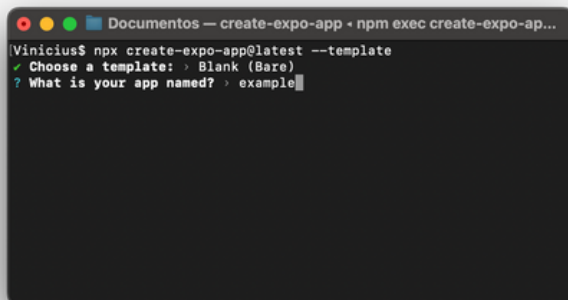
```
npx create-expo-app@latest --template
```

Você terá uma saída como a seguinte:



```
Vinicius.Agile — create-expo-app • npm exec create-expo-ap...
? Choose a template: > - Use arrow-keys. Return to submit.
  Default
  Blank
  Blank (TypeScript)
  Navigation (TypeScript)
> Blank (Bare)
  blank app with the native code exposed (expo prebuild)
```

Escolha a opção "Blank (Bare)". Em seguida, será perguntado sobre qual nome você deseja dar ao seu app, para este exemplo nomeamos nosso app como example.



```
Documentos — create-expo-app • npm exec create-expo-ap...
Vinicius$ npx create-expo-app@latest --template
✓ Choose a template: > Blank (Bare)
? What is your app named? > example
```

Após isso, temos nosso projeto disponível para trabalharmos. Mas antes de seguir podemos analisar as versões atuais do nosso projeto. Para isso, abra seu projeto no Visual Studio Code e abra o package.json.

Para este ebook, iremos trabalhar com as seguintes versões:

expo: ~51.0.28

react: 18.2.0

react-native: 0.74.5



```
1  {
2    "name": "example",
3    "version": "1.0.0",
4    "main": "index.js",
5    "scripts": {
6      "start": "expo start --dev-client",
7      "android": "expo run:android",
8      "ios": "expo run:ios",
9      "web": "expo start --web"
10   },
11   "dependencies": {
12     "expo": "~51.0.28",
13     "expo-status-bar": "~1.12.1",
14     "react": "18.2.0",
15     "react-native": "0.74.5"
16   },
17   "devDependencies": {
18     "@babel/core": "^7.20.0"
19   },
20   "private": true
21 }
22
```

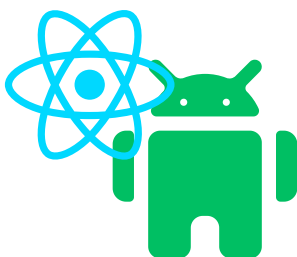
Caso as versões do projeto em que você criou estejam diferentes, não se preocupe. O que você aprenderá neste ebook servirá para versões posteriores e até mesmo para versões anteriores. A intenção de mostrar as versões é apenas para fins didáticos, de modo que você possa comparar com as suas.

Agora que temos nosso app criado. Vamos para o que interessa.

Let's Go!

Módulo Nativo

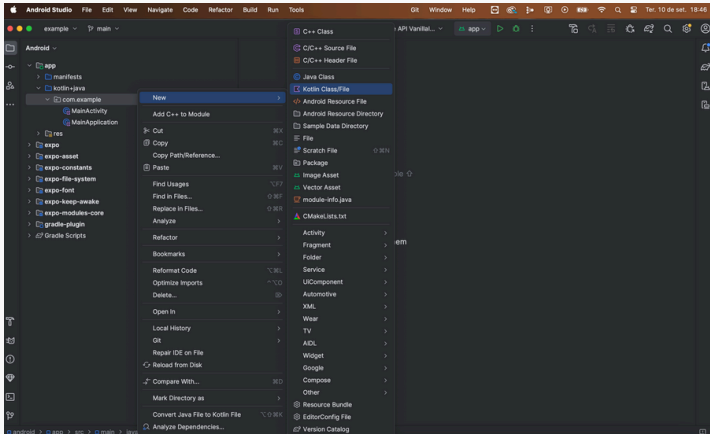
CAPÍTULO 2



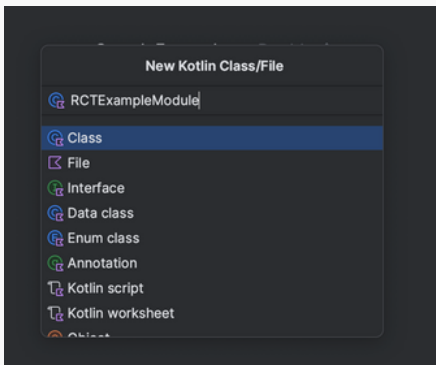
Bridge Android

7. MÓDULO NATIVO ANDROID

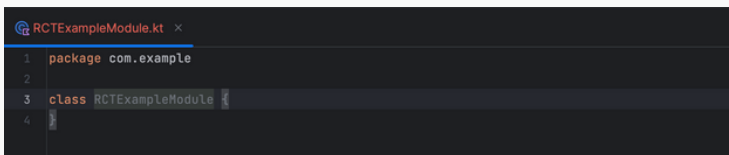
1. Abra a pasta android da raíz do seu projeto no Android Studio.
2. Dentro do Android Studio, navegue em `app/kotlin+java` em `com.SEU_PROJETO` clique com o botão direito, escolha "New" e em seguida escolha a opção "Kotlin Class/File". Veja o caminho na imagem abaixo:



3. Na tela que se abre dê o mesmo nome do arquivo que você nomeou no iOS, no nosso caso, `RCTExampleModule`. Em seguida escolha a opção `Class`.



Você verá um arquivo semelhante a este abaixo:



7.1 CRIANDO CLASSE

Vamos modificar o nosso arquivo `RCTExampleModule.kt`. Vamos fazer o mesmo como fizemos com o iOS, vamos criar a classe que receberá nossas funções. Para isso, escreva o código abaixo em `RCTExampleModule.kt`:

`RCTExampleModule.kt`

```
RCTExampleModule.kt
1 package com.example
2
3 import com.facebook.react.bridge.ReactApplicationContext
4 import com.facebook.react.bridge.ReactContextBaseJavaModule
5
6 class RCTExampleModule(reactContext: ReactApplicationContext) : ReactContextBaseJavaModule(reactContext) {
7     override fun getName() = "RCTExampleModule"
8 }
9
```

Vamos entender o que fizemos:

imports

Esses imports permitem que o módulo Kotlin faça parte do ecossistema React Native, expondo funções nativas e acessando o contexto da aplicação, o que é crucial para o funcionamento de qualquer integração entre código nativo e o React Native.

`class RCTExampleModule(reactContext: ReactApplicationContext) : ReactContextBaseJavaModule(reactContext) {`

`RCTExampleModule`: É uma classe Kotlin que herda de `ReactContextBaseJavaModule`. Isso significa que ela é um módulo que pode ser acessado do código JavaScript no React Native.

`ReactContextBaseJavaModule`: É a classe base para criar módulos nativos em React Native para Android. Ele fornece o contexto da aplicação React Native, necessário para acessar recursos como a API de eventos, ambiente de execução e outras funcionalidades do React Native.

`reactContext`: `ReactApplicationContext`: O contexto é passado como parâmetro para o construtor da classe, permitindo que o módulo acesse o contexto global do React Native.

`override fun getName() = "RCTExampleModule"`

Este método sobrescreve a função `getName` da classe `ReactContextBaseJavaModule`. Ele retorna o nome do módulo, que é como o React Native identificará e chamará o módulo nativo do código JavaScript. Nesse caso, o módulo será identificado como `"RCTExampleModules"`.

É importante aqui definirmos exatamente o mesmo nome que usamos no iOS e vice versa. Isso é essencial para não precisarmos criar validações de chamadas do lado React Native.

7.2 CRIANDO FUNÇÃO

Agora que nossa classe foi criada, podemos trabalhar com a criação de nossa função. Vamos criar uma função simples, apenas printar no terminal uma mensagem. Adicione o código em destaque abaixo:

RCTExampleModule.kt

```
1 package com.example
2
3 import com.facebook.react.bridge.ReactApplicationContext
4 import com.facebook.react.bridge.ReactContextBaseJavaModule
5 import com.facebook.react.bridge.ReactMethod // Adicione esta linha
6 import android.util.Log // Adicione esta linha
7
8 class RCTExampleModule(reactContext: ReactApplicationContext) : ReactContextBaseJavaModule(reactContext) {
9     override fun getName() = "RCTExampleModule"
10
11     // Adicione esta função
12     @ReactMethod
13     fun printMessage() {
14         Log.d( tag: "RCTExampleModule", msg: "printMessage started with title")
15     }
16 }
17
```

Vamos entender o que fizemos:

@ReactMethod

A anotação `@ReactMethod` indica que este método será acessível pelo JavaScript no React Native. Sem essa anotação, o método permaneceria invisível para o sistema de bridge do React Native.

fun printMessage()

Função kotlin que recebe mostra no terminal uma mensagem.

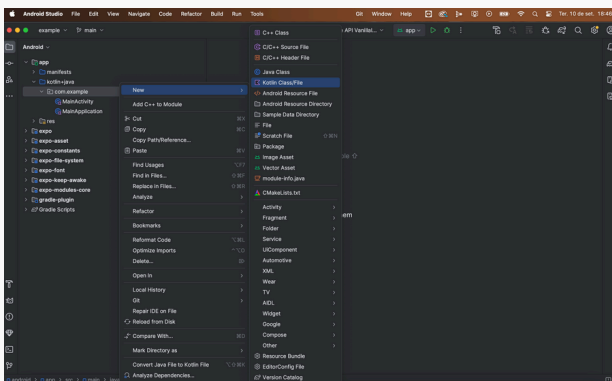
Log.d

Especificamente, imprime uma mensagem de depuração no log do Android com um "tag" (RCTExampleModule) e uma mensagem (printMessage started with title \$title).

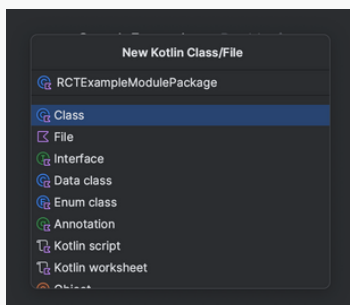
7.3 EXPONDO FUNÇÕES PARA O REACT NATIVE

Uma vez que um módulo nativo é escrito, ele precisa ser registrado com React Native. Para fazer isso, você precisa adicionar seu módulo nativo a um `ReactPackage` e registrar o `ReactPackage` com React Native. Durante a inicialização, o React Native fará um loop em todos os pacotes e, para cada `ReactPackage`, registrará cada módulo nativo dentro.

1. Novamente, navegue em `app/kotlin+java` em `com.SEU_PROJETO` clique com o botão direito, escolha "New" e em seguida escolha a opção "Kotlin Class/File". Veja o caminho na imagem abaixo:



2. Na tela que se abre dê o mesmo nome do arquivo que você nomeou anteriormente, mas ao final, adicione `Package`. Em seguida escolha `Class`.



Ao final você verá algo como abaixo:

```
RCTExampleModulePackage.kt x
1 package com.example
2
3 class RCTExampleModulePackage {
4 }
```

3. Para expor nossa classe, basta adicionar o código abaixo em `deuRCTExampleModulePackage.kt`:

`RCTExampleModulePackage.kt`

```
1 package com.example
2
3 import android.view.View
4 import com.facebook.react.ReactPackage
5 import com.facebook.react.bridge.NativeModule
6 import com.facebook.react.bridge.ReactApplicationContext
7 import com.facebook.react.uimanager.ReactShadowNode
8 import com.facebook.react.uimanager.ViewManager
9
10 class RCTExampleModulePackage : ReactPackage {
11     override fun createViewManagers(
12         reactContext: ReactApplicationContext
13     ): MutableList<ViewManager<View, ReactShadowNode<*>>> = mutableListOf()
14
15     override fun createNativeModules(
16         reactContext: ReactApplicationContext
17     ): MutableList<NativeModule> = listOf(RCTExampleModule(reactContext)).toMutableList()
18 }
19
```

Vamos entender o que fizemos:

imports

São todas as biblioteca e ferramentas que serão necessárias para uso da nossa classe.

class RCTExampleModulePackage : ReactPackage

RCTExampleModulePackage: Esta é a classe que implementa a interface `ReactPackage`, que é um contrato obrigatório para que o React Native possa registrar módulos nativos.

ReactPackage: Interface que deve ser implementada para fornecer ao React Native uma coleção de módulos nativos e gerenciadores de visualizações (View Managers). Sempre que você cria um módulo nativo, ele deve ser registrado dentro de um pacote como este.

createViewManagers

Este método é usado para registrar view managers no React Native. Um view manager gerencia componentes visuais nativos que você deseja expor ao JavaScript.

createNativeModules

Esta função é responsável por registrar os módulos nativos que estarão disponíveis para o React Native. Neste caso, é aqui que o módulo `RCTExampleModule` é adicionado.

O React Native utiliza o método `createNativeModules` para descobrir quais módulos nativos estão disponíveis.

Quando o React Native inicializa o código nativo, ele acessa essa classe

`RCTExampleModulePackage`, que registra o módulo `RCTExampleModule` no ambiente JavaScript. Módulos nativos são, então, acessíveis do lado do JavaScript, possibilitando a chamada de métodos nativos diretamente da aplicação React Native.

MainApplication.kt

```
1 package com.example
2
3 > import ...
4
5 class MainApplication : Application(), ReactApplication {
6
7     override val reactNativeHost: ReactNativeHost = ReactNativeHostWrapper(
8         application: this,
9         object : DefaultReactNativeHost(application: this) {
10             override fun getPackages(): List<ReactPackage> {
11                 // Packages that cannot be autolinked yet can be added manually here, for example:
12                 // packages.add(new MyReactNativePackage());
13                 return PackageList(this).packages
14             }
15
16             override fun getJSMainModuleName(): String = ".expo/.virtual-metro-entry"
17
18             override fun getUseDeveloperSupport(): Boolean = BuildConfig.DEBUG
19
20             override val isNewArchEnabled: Boolean = BuildConfig.IS_NEW_ARCHITECTURE_ENABLED
21             override val isHermesEnabled: Boolean = BuildConfig.IS_HERMES_ENABLED
22         }
23     )
24 }
```

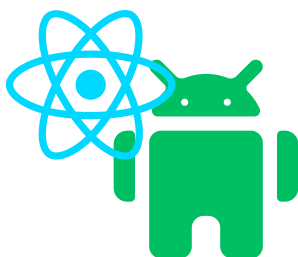
4. Por fim, em MainApplication.kt, vamos substituir esse trecho de código acima pelo código abaixo:

MainApplication.kt

```
1 package com.example
2
3 > import ...
4
5 class MainApplication : Application(), ReactApplication {
6
7     override val reactNativeHost: ReactNativeHost = ReactNativeHostWrapper(
8         application: this,
9         object : DefaultReactNativeHost(application: this) {
10             override fun getPackages(): List<ReactPackage> =
11                 PackageList(reactNativeHost: this).packages.apply { this: ArrayList<ReactPackage?> {
12                     add(RCTExampleModulePackage())
13                 }
14             }
15
16             override fun getJSMainModuleName(): String = ".expo/.virtual-metro-entry"
17
18             override fun getUseDeveloperSupport(): Boolean = BuildConfig.DEBUG
19
20             override val isNewArchEnabled: Boolean = BuildConfig.IS_NEW_ARCHITECTURE_ENABLED
21             override val isHermesEnabled: Boolean = BuildConfig.IS_HERMES_ENABLED
22         }
23     )
24 }
```

Conexão React Native

CAPÍTULO 3



Bridge Android

3. ACESSANDO MÓDULO NATIVO

Finalmente, com nosso código nativo desenvolvido podemos acessar através do React Native. Para isso, iremos precisar usar o `NativeModules` do React Native para fazer essa bridge com nosso código nativo.

Em nosso projeto React Native, acesse o arquivo `App.js` na raiz do projeto, e adicione o código abaixo:

App.js

```
App.js > ...
You, 7 minutes ago | 1 author (You)
1 import React from 'react';
2 import { StyleSheet, Text, View, NativeModules, Button } from 'react-native';
3
4 export default function App() {
5   const { ExampleModule } = NativeModules;
6   return (
7     <View style={styles.container}>
8       <Text>Native module app!</Text>
9       <Button title="Click me for printMessage" onPress={() => ExampleModule.printMessage()} />
10     </View>
11   );
12 }
13
14 const styles = StyleSheet.create({
15   container: {
16     flex: 1,
17     backgroundColor: '#fff',
18     alignItems: 'center',
19     justifyContent: 'center',
20   },
21 });
```

Vamos entender o que fizemos:

`const { ExampleModule } = NativeModules;`

NativeModules: É um objeto fornecido pelo React Native que contém todos os módulos nativos expostos ao JavaScript. Ele permite que o código JavaScript acesse e interaja com funcionalidades nativas do sistema operacional, que foram registradas via bridge.

{ ExampleModule }: Utiliza destructuring assignment para extrair o módulo nativo chamado `ExampleModule` de dentro de `NativeModules`. Nesse caso, `ExampleModule` é o nome que foi registrado anteriormente (via Swift e Objective-C no iOS ou via Kotlin/Java no Android).

No JavaScript, o módulo é acessado como `ExampleModule` devido a essa convenção, simplificando a nomenclatura e garantindo que o código JavaScript fique mais limpo e fácil de ler.

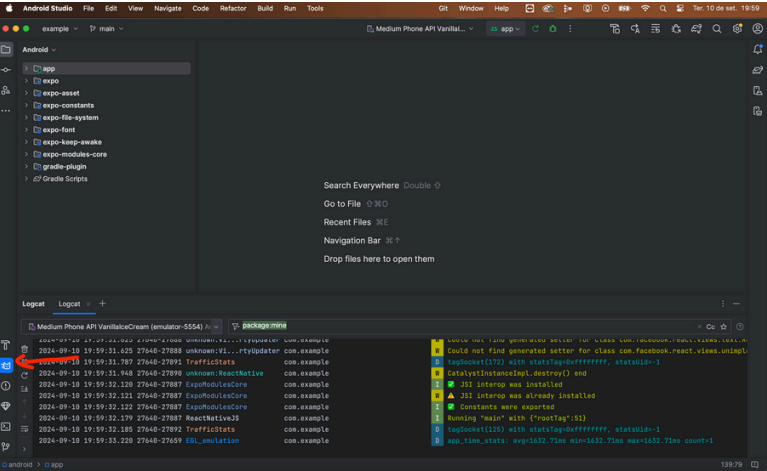
ExampleModule.printMessage()

printMessage: Função kotlin criada e exposta. Conforme criamos, ela será responsável por imprimir no terminal do Android Studio uma mensagem.

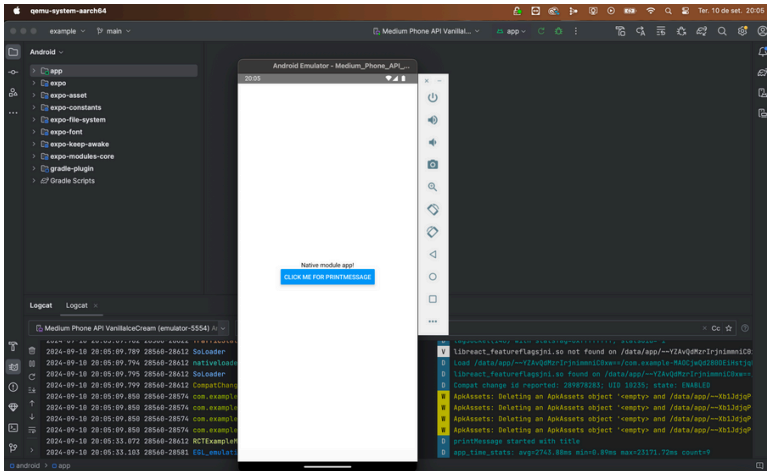
3.1 TESTANDO MODIFICAÇÕES

Finalmente chegou o momento mais esperado do nosso projeto. Vamos testar o que fizemos

- 1. No terminal do VSCode execute o comando `yarn android` ou `npm run android`
- 2. Após rodar o app, com o Android Studio aberto, abra o Logcat. Opção que a seta aponta. Aqui você verá todos os logs do app em tempo de execução

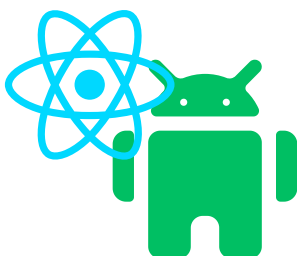


- 3. Pressione o botão "Click me for printmessage". No terminal você verá a mensagem "printMessage started with title".



Envio de Parâmetros

CAPÍTULO 4



Bridge Android

4 ENVIO DE PARÂMETROS

O envio de parâmetros é algo essencial em uma bridge, aliás, é de suma importância sabermos enviar parâmetros pois será dessa forma que poderemos manipular e transportar dados Javascript para a linguagem nativa Kotlin. Assim, poderemos via Javascript controlar nativamente nosso aplicativo e acessar/controlar recursos nativos complexos.

1. No Android, as modificações são bem mais simples que no iOS, bastando apenas alterar o arquivo RCTExampleModule.kt. Altere o seu código com as linhas em destaque:

RCTExampleModule.kt

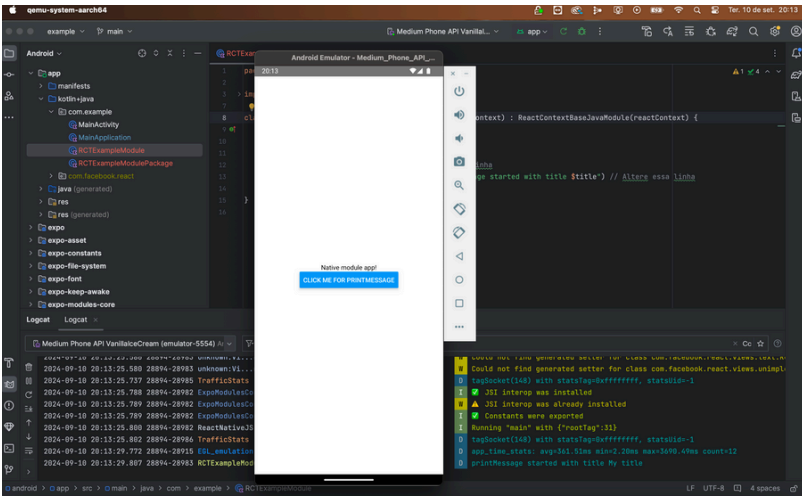
```
@RCTExampleModule.kt x
1 package com.example
2
3 import com.facebook.react.bridge.ReactApplicationContext
4 import com.facebook.react.bridge.ReactContextBaseJavaModule
5 import com.facebook.react.bridge.ReactMethod
6 import android.util.Log
7
8 class RCTExampleModule(reactContext: ReactApplicationContext) : ReactContextBaseJavaModule(reactContext) {
9     override fun getName() = "RCTExampleModule"
10
11     @ReactMethod
12     fun printMessage(title: String) { // Altere essa linha
13         Log.d(log, "RCTExampleModule", msg, "printMessage started with title $title") // Altere essa linha
14     }
15 }
16
```

2. Agora em App.js basta enviar seu texto como parâmetro

App.js

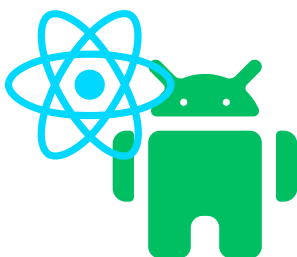
```
App.js > ...
You, 7 minutes ago | 1 author (You)
1 import React from 'react';
2 import { StyleSheet, Text, View, NativeModules, Button } from 'react-native';
3
4 export default function App() {
5     const { ExampleModule } = NativeModules;
6
7     return (
8         <View style={styles.container}>
9             <Text>Native module app!</Text>
10             <Button
11                 title="Click me for printMessage"
12                 onPress={() => ExampleModule.printMessage("My title")}
13             />
14         </View>
15     );
16 }
17
18 const styles = StyleSheet.create({
19     container: {
20         flex: 1,
21         backgroundColor: '#fff',
22         alignItems: 'center',
23         justifyContent: 'center',
24     },
25 });
26
```

Você verá a saída "printMessage started with title My title" no LogCat:



Retorno de Função

CAPÍTULO 5



Bridge Android

5 RETORNO DE FUNÇÃO

O ideal de uma conexão é que seja bidirecional, ou seja, ela envia informações e também recebe informações. Isso nos ajuda a criar aplicações e funcionalidades complexas e robustas. Uma bridge também deve conter uma comunicação bidirecional. E é sobre isso que iremos aprender nesse capítulo

1. Vamos então agora realizar novas modificações no que fizemos até o momento. Vamos começar pelo `RCTExampleModule.kt`. Adicione os códigos em destaque:

`RCTExampleModule.kt`

```
1 package com.example
2
3 import com.facebook.react.bridge.ReactApplicationContext
4 import com.facebook.react.bridge.ReactContextBaseJavaModule
5 import com.facebook.react.bridge.ReactMethod
6 import android.util.Log
7 import com.facebook.react.bridge.Promise // Adicione esta linha de código
8
9 class RCTExampleModule(reactContext: ReactApplicationContext) : ReactContextBaseJavaModule(reactContext) {
10     override fun getName() = "RCTExampleModule"
11
12     @ReactMethod
13     fun printMessage(title: String) {
14         Log.d(tag="RCTExampleModule", msg="printMessage started with title $title")
15     }
16
17     // Adicione esta função
18     @ReactMethod
19     fun returnMessage(title: String, promise: Promise) {
20         promise.resolve("Message processed: $title")
21     }
22 }
23
```

Vamos entender o que fizemos:

`returnMessage`

Função que recebe dois parâmetros:

title: String: Este parâmetro recebe uma string enviada do código JavaScript, que pode ser usada no processamento dentro da função.

promise: Promise: O parâmetro promise é um objeto especial fornecido pelo React Native para funções que realizam operações assíncronas ou precisam retornar valores ao JavaScript. Esse Promise permite que a função envie de volta um resultado ou um erro.

```
promise.resolve("Message processed: $title")
```

Esta chamada resolve o Promise, retornando o valor para o JavaScript

O método resolve é chamado quando a operação é bem-sucedida, e o valor passado para ele será retornado ao código JavaScript.

Neste caso, o valor que está sendo retornado é uma string concatenada:

"Message processed: \$title", onde title é o valor recebido como parâmetro.

Isso significa que a string enviada do React Native é processada e devolvida com uma mensagem de confirmação.

3. Agora em nosso App.js podemos realizar a alteração que precisamos para capturar o retorno de uma chamada. Adicione os códigos em destaque.

App.js

```
App.js
1 import { Platform, StatusBar, StyleSheet, View } from 'react-native';
2
3 export default function App() {
4   const { ExampleModule } = NativeModules;
5
6   return (
7     <View style={styles.container}>
8       <TextNative module={ExampleModule} />
9     </View>
10  );
11
12  title="Click on for printMessage"
13  onPress={() => ExampleModule.printMessage('My title')}
14
15  // Adicione este trecho de código <+>
16  <Button title="Click on for returnMessage" onPress={() =>
17    ExampleModule.returnMessage('My title').then(result) => {
18      console.log('Mensagem recebida nativa', result);
19    }
20  } />
21  </Button>
22  </View>
23  </View>
24  </View>
25  </View>
26
27  const styles = StyleSheet.create({
28    container: {
29      flex: 1,
30      backgroundColor: 'black',
31      alignItems: 'center',
32      justify-content: 'center',
33    },
34  });
35
36 }
```

Vamos entender o que fizemos:

ExampleModule.returnMessage('My title')

ExampleModule: Refere-se ao módulo nativo que foi exposto pelo sistema de bridge do React Native. Esse módulo foi importado usando NativeModules e corresponde à classe Kotlin RCTExampleModule.

returnMessage('My title'): Está chamando o método returnMessage do módulo nativo, passando a string 'My title' como argumento. No código nativo, essa string é recebida como um parâmetro (title) e processada. O método returnMessage retorna uma Promise para o código JavaScript.

.then((result) => { ... })

.then(): É parte da estrutura de Promises em JavaScript. Ele define o que acontecerá quando a Promise for resolvida com sucesso. Isso significa que o código Kotlin executou o método sem erros e retornou um resultado.

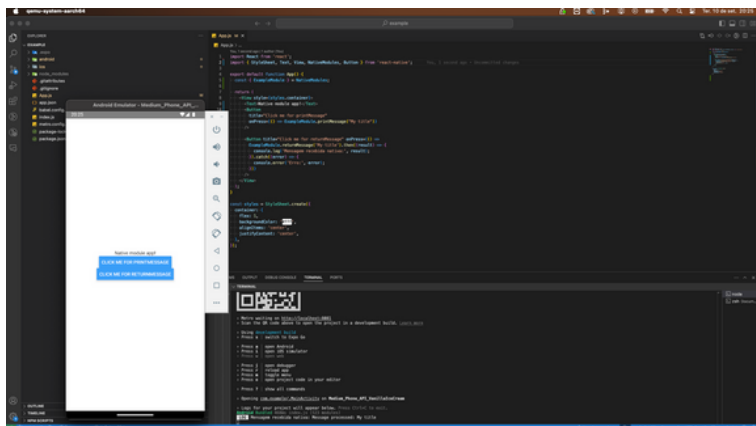
(result) => { console.log('Mensagem recebida nativa:', result); } Essa função é executada quando a Promise é resolvida. O parâmetro result contém o valor retornado pelo método nativo Kotlin. Nesse caso, a função exibe no console o texto "Mensagem recebida nativa:" seguido do resultado retornado pelo Kotlin.

.catch((error) => { ... })

.catch(): Também faz parte da estrutura de Promises. Ele define o que acontecerá se a Promise for rejeitada, ou seja, se ocorrer algum erro ao executar o método nativo.

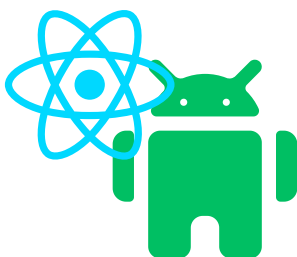
(error) => { console.error('Erro:', error); } Essa função é chamada quando a Promise é rejeitada. O parâmetro error contém detalhes do erro que ocorreu. Nesse caso, ele exibe no console a mensagem "Erro:" seguida do conteúdo do erro.

Após rodar o seu app novamente e pressionar o botão "Click me for returnMessage" você verá no terminal do VSCode a mensagem "*Mensagem recebida nativa: Message processed: My title*". Esta mensagem é a concatenação de "*Mensagem recebida nativa:*" (log da linha 17) com "*Message processed:* " (Texto nativo da função Kotlin) e "*My title*" (Parâmetro enviado para nossa função nativa)



Event Listener

CAPÍTULO 6



Bridge Android

6 RETORNO EVENT LISTENER

Eventos listeners são também uma ótima forma de trabalharmos com bridge, isso porque podemos manipular dados assíncronos de forma independente em nosso código nativo enquanto ouvimos e aguardamos suas alterações do lado React Native. Vamos aprender como podemos fazer isso.

1. Vamos começar pelo RCTExampleModule.kt.

Adicione o seguinte código em destaque:

RCTExampleModule.kt

```
1 package com.example
2
3 import com.facebook.react.bridge.ReactApplicationContext
4 import com.facebook.react.bridge.ReactContextBaseJavaModule
5 import com.facebook.react.bridge.ReactMethod
6 import android.util.Log
7 import com.facebook.react.bridge.Promise
8 import com.facebook.react.bridge.Arguments // Adicione esta linha de código
9 import com.facebook.react.modules.core.DeviceEventManagerModule // Adicione esta linha de código
10
11 class RCTExampleModule(reactContext: ReactApplicationContext) : ReactContextBaseJavaModule(reactContext) {
12     override fun getName() = "RCTExampleModule"
13
14     @ReactMethod
15     fun printMessage(title: String) {
16         Log.d(LOG, "RCTExampleModule", "Msg: 'printMessage started with title $title'")
17     }
18
19     @ReactMethod
20     fun returnMessage(title: String, promise: Promise) {
21         promise.resolve("Message processed: $title")
22     }
23
24     // Adicione esta função
25     @ReactMethod
26     fun eventMessage(value: Int) {
27         val params = Arguments.createMap()
28         params.putInt("value", value)
29         sendEvent("onMessagePrinted", params)
30     }
31
32     // Adicione esta função
33     private fun sendEvent(eventName: String, params: Any?) {
34         reactApplicationContext
35             .getJSModule(DeviceEventManagerModule.RCTDeviceEventEmitter::class.java)
36             .emit(eventName, params)
37     }
38 }
```

Vamos entender o que fizemos:

eventMessage

title: Int: Essa função recebe um parâmetro title, que é uma número enviado do JavaScript.

Arguments.createMap(): Cria um mapa de argumentos (semelhante a um objeto JSON) que será enviado como o corpo do evento. O método createMap() retorna um objeto do tipo WritableMap. Isso é necessário para para capturar os valores da mesma forma como fazemos para iOS.

params.putString("value", value): Adiciona uma chave "value" e seu respectivo valor (value) ao mapa. Esse valor será enviado ao JavaScript como parte do evento.

sendEvent("onMessagePrinted", params): Chama a função sendEvent para emitir um evento com o nome "onMessagePrinted" e passar os parâmetros criados no mapa.

2. Vamos começar pelo RCTExampleModule.kt.

Adicione o seguinte código em destaque:

App.js

```
App.js > App
1  import React from 'react';
2  import { StyleSheet, Text, View, NativeModules, Button, NativeEventEmitter } from 'react-native';
3
4  export default function App() {
5    const { ExampleModule } = NativeModules;
6    const eventEmitter = new NativeEventEmitter(ExampleModule); // Adicione esta linha
7
8    // Adicione esta função
9    const subscription = eventEmitter.addListener('onMessagePrinted', (event) => {
10      console.log("Evento recebido:", event.value);
11    });
12
13    return (
14      <View style={styles.container}>
15        <Text Native Module app!></Text>
16        <Button
17          title="Click me for printMessage"
18          onPress={() => ExampleModule.printMessage("My title")}
19        />
20
21        <Button title="Click me for returnMessage" onPress={() =>
22          ExampleModule.returnMessage("My title").then(result) => {
23            console.log("Message recebida nativa", result);
24          }).catch(error) => {
25            console.error('Error:', error);
26          }
27        />
28      />
29      /* Adicione este trecho de código */ You, 1 second ago • Uncommitted changes
30    <Button
31      title="Click me for eventMessage"
32      onPress={() => ExampleModule.eventMessage(3)}
33    />
34  </View>
35  );
36  }
37
38  const styles = StyleSheet.create({
39    container: {
40      flex: 1,
41      backgroundColor: 'white',
42      alignItems: 'center',
43      justifyContent: 'center',
44    },
45  });
```

Vamos entender o que fizemos:

```
const eventEmitter = new NativeEventEmitter(ExampleModule);
```

Cria uma instância de NativeEventEmitter associada ao ExampleModule. Isso permite que o JavaScript escute eventos emitidos por ExampleModule.

```
const subscription
```

eventEmitter.addListener('onMessagePrinted', ...): Adiciona um listener que escuta eventos com o nome "onMessagePrinted" emitidos pelo ExampleModule.

(event) => { console.log("Evento recebido:", event.value) }: Função callback que é chamada sempre que o evento "onMessagePrinted" é emitido. Ela simplesmente imprime o valor recebido no console.

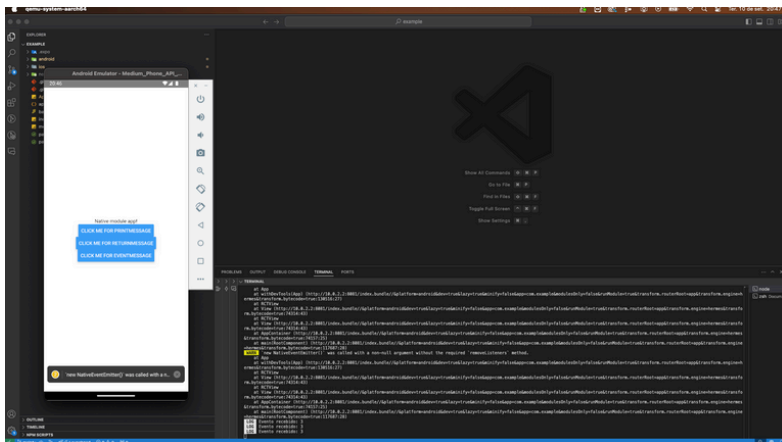
```
ExampleModule.eventMessage(3)
```

Dispara nossa função nativa eventMessage enviando como parâmetro o valor numérico 3.

Importante: Não esqueça de remover o listener quando não for mais necessário ouvir modificações. Isso é importante para evitar problemas de performance do seu aplicativo.

Para remover o listener basta usar o comando `subscription.remove()`.

2. Após rodar novamente o app, e clicar no botão "Click me for eventMessage" você verá a saída "Evento recebido: 3". Esse é o retorno do listener que estamos escutando em na constant *subscription*.



Note que apesar do retorno, o React Native nos alerta de um erro. Isso ocorre porque o NativeEventEmitter espera que o módulo nativo forneça a implementação dos métodos `addListener` e `removeListeners` para gerenciar corretamente os ouvintes de eventos no lado do JavaScript. Esses métodos são necessários para evitar vazamento de memória ao adicionar e remover listeners de eventos.

Vamos então fazer pequenas modificações para resolver o problema. Em `RCTExampleModule.kt` adicione os trechos em destaque.

RCTExampleModule.kt

```
1 package com.example
2
3 import com.facebook.react.bridge.ReactApplicationContext
4 import com.facebook.react.bridge.ReactContextBaseJavaModule
5 import com.facebook.react.bridge.ReactMethod
6 import android.util.Log
7 import com.facebook.react.bridge.Promise
8 import com.facebook.react.bridge.Arguments
9 import com.facebook.react.modules.core.DeviceEventManagerModule
10
11 class RCTExampleModule(reactContext: ReactApplicationContext) : ReactContextBaseJavaModule(reactContext) {
12     private var listenerCount = 0 // Adicione esta linha de código
13     override fun getName() = "RCTExampleModule"
14
15     @ReactMethod
16     fun printMessage(title: String) {}
17
18     @ReactMethod
19     fun returnMessage(title: String, promise: Promise) {}
20
21     @ReactMethod
22     fun eventMessage(value: Int) {}
23
24     private fun sendEvent(eventName: String, params: Any?) {}
25
26     // Adicione este método
27     @ReactMethod
28     fun addListener(eventName: String) {
29         listenerCount ++ 1
30     }
31
32     // Adicione este método
33     @ReactMethod
34     fun removeListeners(count: Int) {
35         listenerCount -= count
36     }
37 }
```

[illegible]

Resumo

CAPÍTULO 7

7 RESUMO

Neste ebook, fornecemos uma base sólida para conectar React Native com código nativo em Android. Você aprendeu a criar funções nativas, enviar e receber dados, e gerenciar eventos, o que é crucial para construir aplicações móveis robustas e interativas. Com essas habilidades, você está apto a integrar e utilizar o código nativo de forma eficiente, ampliando as possibilidades do seu desenvolvimento com React Native.

Agradecemos por acompanhar este guia e esperamos que as técnicas e conceitos apresentados aqui sejam valiosos para seus projetos futuros. Continue explorando e inovando com React Native e Android!